JStall

Record & Replay + IntelliJ Plugin

New in 0.5.x · How recording works · IDE integration

github.com/parttimenerd/jstall

What's New Since 0.4.11

Feature	Details
Record & Replay	Capture thread dumps + flamegraph into a self-contained ZIP
`all` target	Run any command against every discovered JVM in parallel
`--intelligent-filter`	Regex-based frame collapsing for Spring, Netty, gRPC, JDK internals
`compiler-queue`	Active JIT compilations + queued tasks via `jcmd Compiler.queue`
`gc-heap-info`	Heap info snapshots + deltas via `jcmd GC.heap_info`
`vm-vitals` / `vm-metaspace`	VM telemetry from `jcmd` diagnostic commands
`processes`	Non-JVM processes consuming high CPU (`/proc` , `ps`)

Record: Capturing Diagnostics

One single command to capture all:

```
jstall record <pid-or-filter> --output out.zip
```

Collects thread dumps, system properties, JFR, flamegraph, and jcmd diagnostics – all in one pass, packed into a self-contained ZIP.

The Recording ZIP

```
out.zip:
out
├── metadata.json                              # recording metadata
├── README.md                                  # human-readable summary (see next slide)
└── <pid>/                                     # per-JVM directory (by PID)
    ├── manifest.json                          # JVM manifest
    ├── thread-dumps/
    │   ├── 000-<timestamp>.txt                # jcmd Thread.print snapshots
    │   └── 001-<timestamp>.txt
    ├── flamegraphs/
    │   ├── flame.html                         # async-profiler HTML
    │   └── flame.meta.json
    ├── jfr/default.jfr                        # JFR recording
    ├── gc-heap-info/000-….txt, 001-….txt      # jcmd GC.heap_info snapshots
    ├── vm-metaspace/000-….txt, 001-….txt      # jcmd VM.metaspace snapshots
    ├── vm-classloader-stats/000-….txt, …      # jcmd VM.classloader_stats
    ├── compiler-queue/000-….txt, …            # jcmd Compiler.queue snapshots
    ├── VM_vitals/data.txt                     # jcmd VM.vitals
    ├── vm-info/data.txt                       # jcmd VM.info
    ├── vm-command-line/data.txt               # jcmd VM.command_line
    ├── vm-flags/data.txt                      # jcmd VM.flags
    ├── vm-uptime/data.txt                     # jcmd VM.uptime
    ├── system-properties/data.txt             # jcmd VM.system_properties
    └── system-environment/000-….json, …       # system environment snapshots
```

The Recording README

Each ZIP includes an auto-generated `README.md`:

```
# JStall Recording Archive

Created by `jstall record` v0.4.11 on **2026-03-12 16:02:18 UTC**.

Project: <https://github.com/parttimenerd/jstall>

## Table of Contents

- [Recording Configuration](#recording-configuration)
- [JVM 67242 — Main](#jvm-67242)
- [Usage](#usage)
...

## JVM 67242 — Main

| Property | Value |
|---|---|
| PID | 67242 |
| Main class | `Main` |
| Java version | 26-ea |
| Vendor | Oracle Corporation |
| Release date | 2026-03-17 |
```

Live Demo: Record → Replay

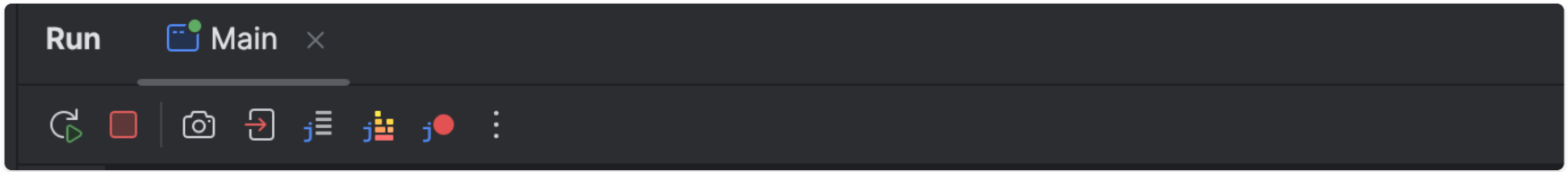
```
# 1. Record a running application
jstall record <pid> --output demo.zip

# 2. Replay: status analysis on the recording
jstall status demo.zip

# 3. Replay: check for deadlocks
jstall deadlock demo.zip

# 4. Open the flamegraph directly
open demo.zip  # extract flamegraph.html and view in browser
```

IntelliJ Plugin – Core Ideas

- **Thin wrapper** – shells out to JStall CLI, no separate analysis engine
- **Auto PID detection** – extracts PID from the current Run tab
- **Recordings as first-class files** – custom icon, double-click to analyze, right-click for flamegraph
- **Embedded flamegraph viewer** – JCEF browser inside IntelliJ

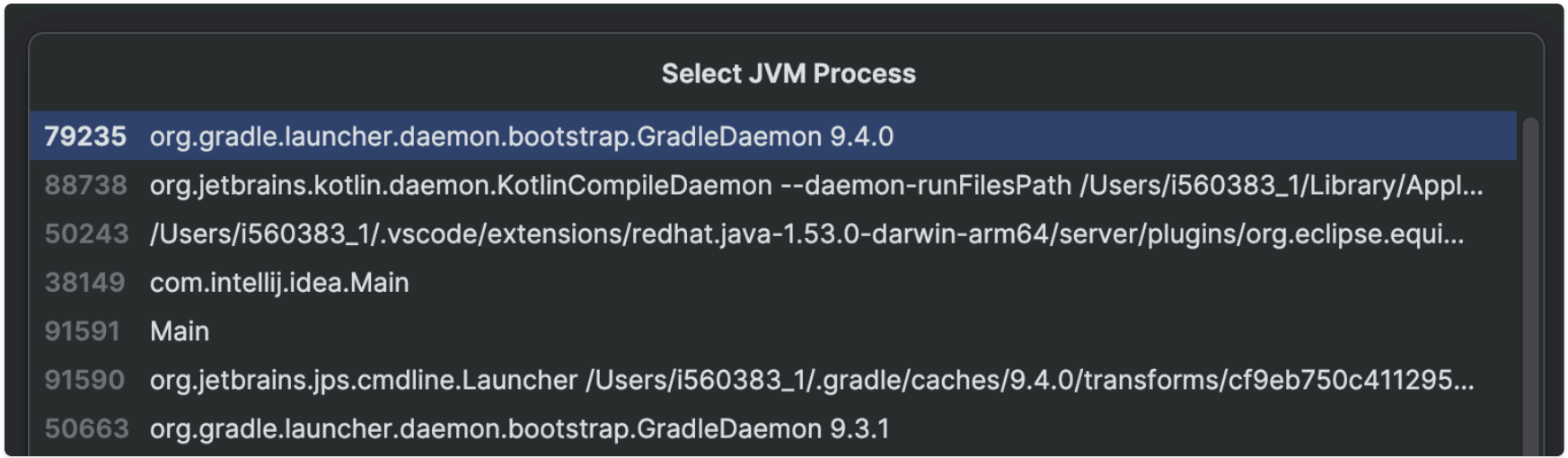
Three toolbar actions: **Status** · **Record** · **Flame**

Plugin: Toolbar



Status / Record / Flame actions in the Run tool window toolbar. Auto-detects PID from the current Run tab.

Plugin: JVM Selector



Fallback JVM picker popup when PID can't be auto-detected – lists processes via `jcmd`.

Plugin: Status Output

Plugin: Flamegraph Viewer

Embedded JCEF browser. Each profiling run gets its own tab. Gear menu → save as standalone HTML.

Plugin: Recording Files in the IDE

Recording ZIPs are first-class objects in the project view:

- **Custom file icon** – visually distinct from regular ZIPs
- **Double-click** → runs `jstall status -f <zip>` and opens results in an editor tab
- **Right-click → Analyze JStall Recording** – same as double-click
- **Right-click → Show JStall Flamegraph** – opens flamegraph from ZIP in embedded viewer (only shown when recording contains one)
- **Right-click → Extract JStall Recording** – unpacks the ZIP into a folder

The plugin reads the flamegraph directly from the ZIP – no temp files, no extraction step.

Live Demo: Plugin + Recordings

```
1. Run a sample app in IntelliJ
2. Click Record → ZIP appears in project dir
3. Double-click the ZIP → status analysis in editor tab
4. Right-click → Show JStall Flamegraph → embedded viewer
5. Click Flame on the live app → compare with recording
```

Links

CLI: `jbang jstall@parttimenerd/jstall <pid>` · Maven Central `me.bechberger:jstall:0.5.3`

Plugin: JetBrains Marketplace → "JStall" (plugin ID `30667`)

Source: github.com/parttimenerd/jstall · github.com/parttimenerd/jstall-intellij-plugin

Questions?